

# SPEC.md — JobMatch (Spec-Driven Development)

Ce document est la **source de vérité** du projet. Tout code produit par un agent doit satisfaire les exigences ci-dessous. En cas de conflit entre le code et ce document, le document fait foi — le code doit être corrigé, pas la spec.

Relation avec les autres documents :

- `README.md` → architecture technique et stack (le "comment" à haut niveau)
- `SPEC.md` (ce fichier) → exigences précises et vérifiables (le "quoi", testable)
- `TICKETS.md` → découpage en tâches (le "quand" / qui fait quoi)

---

## 1. Vision & objectif

Permettre à un utilisateur de déposer son CV et d'obtenir automatiquement une liste d'offres d'emploi pertinentes, classées par score de correspondance, sans intervention manuelle après l'upload.

**Métrique de succès :** pour un CV donné, au moins 70% des offres retournées dans le top 10 doivent contenir au moins une compétence ou un titre de poste présent dans le CV (mesuré sur le jeu de test de validation, section 8).

---

## 2. Non-objectifs (hors scope explicite)

- Pas de scraping de sites interdisant le scraping dans leurs CGU (LinkedIn, Indeed). Seules les APIs officielles de fournisseurs d'offres sont utilisées.
- Pas de matching basé sur un LLM (coût à éviter) — uniquement extraction par règles/OCR et matching lexical (TF-IDF/BM25).
- Pas de candidature automatique aux offres (le système recommande, ne postule pas).
- Pas de multi-langue pour le MVP (français uniquement pour l'extraction de CV et le matching).
- Pas d'application mobile native.

---

## 3. User stories

| ID   | En tant que | Je veux                    | Afin de                  |
|------|-------------|----------------------------|--------------------------|
| US-1 | Utilisateur | Uploader mon CV (PDF/DOCX) | Que le système l'analyse |

---

| ID   | En tant que | Je veux                                                               | Afin de                                                       |
|------|-------------|-----------------------------------------------------------------------|---------------------------------------------------------------|
| US-2 | Utilisateur | Voir les informations extraites de mon CV                             | Corriger d'éventuelles erreurs d'extraction                   |
| US-3 | Utilisateur | Lancer une recherche d'offres basée sur mon CV                        | Obtenir des offres pertinentes sans les chercher manuellement |
| US-4 | Utilisateur | Voir un score de correspondance et sa justification pour chaque offre | Comprendre pourquoi une offre m'est proposée                  |
| US-5 | Utilisateur | Filtrer les résultats par localisation                                | Ne voir que les offres compatibles avec ma situation          |
| US-6 | Utilisateur | Créer un compte et me connecter                                       | Retrouver mes CV et recherches précédentes                    |

## 4. Exigences fonctionnelles (format EARS)

### 4.1 Authentification

- **REQ-AUTH-1** : QUAND un utilisateur soumet un email et un mot de passe valides sur `/api/auth/register`, LE SYSTÈME DOIT créer un compte et retourner un statut `201`.
- **REQ-AUTH-2** : QUAND un utilisateur soumet un email déjà utilisé, LE SYSTÈME DOIT retourner une erreur `409` sans créer de doublon.
- **REQ-AUTH-3** : QUAND un utilisateur se connecte avec des identifiants valides sur `/api/auth/login`, LE SYSTÈME DOIT retourner un token JWT valide 24h.
- **REQ-AUTH-4** : QUAND une requête est faite sur un endpoint protégé sans token valide, LE SYSTÈME DOIT retourner `401`.

### 4.2 CV Parser

- **REQ-CV-1** : QUAND un fichier PDF, DOCX est uploadé sur `/api/cv/upload`, LE SYSTÈME DOIT accepter le fichier, le stocker sur S3/MinIO et déclencher un job de parsing asynchrone.
- **REQ-CV-2** : QUAND un fichier d'un type non supporté (autre que PDF/DOCX) est uploadé, LE SYSTÈME DOIT rejeter la requête avec un statut `422` et un message explicite.
- **REQ-CV-3** : QUAND un fichier dépasse 10 Mo, LE SYSTÈME DOIT rejeter la requête avec un statut `413`.
- **REQ-CV-4** : QUAND le parsing d'un CV texte (PDF natif ou DOCX) est déclenché, LE SYSTÈME DOIT produire un JSON structuré conforme au schéma défini en section 6.1 en moins de 15 secondes.

- **REQ-CV-5** : QUAND un PDF est un CV scanné (pas de texte extractible), LE SYSTÈME DOIT basculer automatiquement sur l'OCR (Tesseract) avant extraction.
- **REQ-CV-6** : QUAND le parsing échoue (fichier corrompu, illisible), LE SYSTÈME DOIT marquer le statut du CV comme `error` avec un message de diagnostic, et NE DOIT PAS bloquer les autres jobs en cours.
- **REQ-CV-7** : QUAND le parsing est terminé, LE SYSTÈME DOIT permettre à l'utilisateur de consulter et corriger manuellement chaque champ extrait via `PATCH /api/cv/{id}/profile`.

### 4.3 Job Providers

- **REQ-PROV-1** : LE SYSTÈME DOIT exposer une interface `JobProvider` commune que chaque source d'offres implémente, sans logique spécifique à une source en dehors de sa propre classe.
- **REQ-PROV-2** : QUAND une recherche est lancée, LE SYSTÈME DOIT interroger tous les providers actifs en parallèle (pas séquentiellement).
- **REQ-PROV-3** : QUAND un provider retourne une erreur ou un timeout (> 10s), LE SYSTÈME DOIT continuer avec les résultats des autres providers et logger l'échec, SANS faire échouer la recherche globale.
- **REQ-PROV-4** : QUAND un provider retourne des offres, LE SYSTÈME DOIT les normaliser au format `JobOffer` (section 6.2) avant stockage.
- **REQ-PROV-5** : QUAND une offre déjà connue (même `source` + `external_id`) est retournée à nouveau, LE SYSTÈME DOIT mettre à jour l'enregistrement existant plutôt que créer un doublon.

### 4.4 Matching Engine

- **REQ-MATCH-1** : QUAND une recherche est déclenchée pour un CV donné, LE SYSTÈME DOIT calculer un score de 0 à 100 pour chaque offre récupérée, basé sur la similarité lexicale (TF-IDF/BM25) entre le profil CV et le texte de l'offre.
- **REQ-MATCH-2** : LE SYSTÈME DOIT appliquer un boost de score mesurable lorsque des compétences extraites du CV apparaissent explicitement dans le texte de l'offre.
- **REQ-MATCH-3** : QUAND le score est calculé, LE SYSTÈME DOIT stocker la liste des mots-clés ayant contribué au match (traçabilité pour justification côté frontend).
- **REQ-MATCH-4** : LE SYSTÈME NE DOIT PAS appeler d'API LLM externe (OpenAI, Anthropic, etc.) dans le pipeline de matching — uniquement du calcul local (TF-IDF/BM25/spaCy).
- **REQ-MATCH-5** : QUAND les résultats sont retournés à l'utilisateur, LE SYSTÈME DOIT les trier par score décroissant.

### 4.5 Recherche & résultats (orchestration API)

- **REQ-SEARCH-1** : QUAND `POST /api/search` est appelé avec un `cv_id` valide, LE

SYSTÈME DOIT créer un job de recherche asynchrone et retourner immédiatement un `search_id` avec statut `202`.

- **REQ-SEARCH-2** : QUAND `GET /api/search/{id}/results` est appelé avant la fin du job, LE SYSTÈME DOIT retourner un statut `pending`.
- **REQ-SEARCH-3** : QUAND le job de recherche est terminé, LE SYSTÈME DOIT retourner la liste des offres triées avec leur score et mots-clés matchés.
- **REQ-SEARCH-4** : LE SYSTÈME DOIT permettre de filtrer les résultats par `location` via un paramètre de requête optionnel.

4.6 Frontend

- **REQ-FE-1** : LE SYSTÈME DOIT permettre l'upload d'un CV par glisser-déposer ou sélection de fichier.
- **REQ-FE-2** : QUAND le parsing est en cours, LE SYSTÈME DOIT afficher un indicateur de statut (`pending/done/error`) mis à jour sans rechargement de page (`polling` ou `websocket`).
- **REQ-FE-3** : LE SYSTÈME DOIT afficher, pour chaque offre du dashboard, le score de match et au moins un mot-clé ayant justifié ce score.
- **REQ-FE-4** : LE SYSTÈME DOIT permettre d'éditer manuellement les champs extraits du CV (compétences, titres, expériences).

5. Exigences non-fonctionnelles

| Catégorie       | Exigence                                                                                                                               |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Performance     | Une recherche complète (tous providers + matching) doit se terminer en moins de 30 secondes pour un CV standard                        |
| Coût            | Aucun appel à une API LLM payante dans le flux principal (parsing + matching)                                                          |
| Disponibilité   | Le système doit rester fonctionnel si un provider externe est indisponible (dégradation gracieuse, cf. REQ-PROV-3)                     |
| Sécurité        | Les mots de passe sont hashés (bcrypt/argon2), jamais stockés en clair. Les fichiers CV sont stockés avec accès restreint (URL signée) |
| Scalabilité     | Les workers (parsing, providers, matching) doivent pouvoir tourner en plusieurs instances sans état partagé en mémoire                 |
| Observabilité   | Chaque job asynchrone (parsing, provider, matching) doit logger son statut et sa durée d'exécution                                     |
| Confidentialité | Un utilisateur ne peut accéder qu'à ses propres CV et résultats de recherche (isolation par <code>user_id</code> )                     |

---

## 6. Contrats de données

### 6.1 CV structuré (sortie du CV Parser)

```
json
{
  "skills": ["Python", "React", "Docker"],
  "titles": ["Développeur Full Stack", "Ingénieur logiciel"],
  "years_experience": 4,
  "languages": ["français", "anglais"],
  "education": [
    {"degree": "Master Informatique", "institution": "...", "year": 2020}
  ],
  "raw_text": "..."}
}
```

Contraintes :

- `skills` : liste de chaînes non vides, dédoublées (insensible à la casse)
- `years_experience` : entier  $\geq 0$ , ou `null` si non déterminable
- `raw_text` : toujours présent, même si l'extraction structurée échoue partiellement (fallback pour matching brut)

### 6.2 Offre normalisée (`JobOffer`)

```
json
{
  "source": "france_travail",
  "external_id": "123456",
  "title": "Développeur Backend Python",
  "company": "Acme Corp",
  "description": "...",
  "location": "Paris, France",
  "url": "https://...",
  "posted_at": "2026-06-01T00:00:00Z",
  "scraped_at": "2026-07-07T10:00:00Z"}
}
```

Contraintes :

- `source` + `external_id` forment une clé unique (idempotence, cf. REQ-PROV-5)
- `url` doit être une URL absolue valide

### 6.3 Résultat de matching (`Match`)

```
json
{
  "cv_id": "uuid",
  "job_offer_id": "uuid",
  "score": 87,
  "matched_keywords": ["Python", "Docker", "4 ans d'expérience"],
  "created_at": "2026-07-07T10:05:00Z"
}
```

Contraintes :

- `score` : entier entre 0 et 100 inclus
  - `matched_keywords` : peut être vide si score très bas, mais jamais absent (liste vide plutôt que `null`)
- 

## 7. Definition of Done (par module)

Un module est considéré "Done" seulement si :

1. Toutes les exigences EARS associées (section 4) sont couvertes par au moins un test automatisé
  2. Le code respecte les contrats de données (section 6) — validation via Pydantic côté backend
  3. Les cas d'erreur (fichier invalide, provider en panne, etc.) sont gérés explicitement, pas seulement le cas nominal
  4. La documentation OpenAPI (`/docs`) reflète les endpoints réellement implémentés
  5. Le ticket Jira correspondant (voir `TICKETS.md`) a ses critères d'acceptation cochés
- 

## 8. Jeu de validation (test d'acceptation global)

Pour valider le système de bout en bout, constituer :

- 5 CV de test représentatifs (2 PDF natifs, 1 PDF scanné, 2 DOCX), couvrant des profils différents (junior/senior, secteurs différents)
- Pour chaque CV, une liste manuelle des offres "attendues comme pertinentes" parmi un échantillon fixe d'offres de test

Critère de passage : sur ce jeu, au moins 70% des offres retournées en top 10 par le système doivent correspondre aux offres jugées pertinentes manuellement (cf. métrique de succès,

section 1).

---

## 9. Hypothèses & contraintes

- Les CV sont rédigés en français.
  - L'utilisateur dispose d'un CV au format PDF ou DOCX (pas d'images seules type photo de CV papier, seulement PDF scanné classique).
  - Les clés API des providers (France Travail, Adzuna, etc.) sont fournies via variables d'environnement et gérées hors de ce dépôt.
  - Le volume attendu en MVP est faible (< 100 utilisateurs simultanés) — pas d'exigence de haute disponibilité multi-région.
- 

## 10. Traçabilité SPEC ↔ TICKETS

| Section SPEC               | Ticket Jira |
|----------------------------|-------------|
| §4.1 Authentification      | TICKET-1    |
| §4.2 CV Parser             | TICKET-2    |
| §4.3 Job Providers         | TICKET-3    |
| §4.4 Matching Engine       | TICKET-4    |
| §4.5 Recherche & résultats | TICKET-5    |
| §4.6 Frontend              | TICKET-6    |

Toute modification d'une exigence dans ce document doit être répercutée dans le ticket Jira correspondant, et inversement — aucun des deux documents ne doit diverger de l'autre.